

```
USE MyDatabase;
select * from customers;
select * from orders;
select * from employees;
```

```
/*
    For each customer, calculate their total number of orders and total sales
    amount. Only include customers
    whose total sales exceed 600. Sort the result by total sales in descending
    order.
*/
```

```
SELECT
    c.id AS customer_id,
    c.first_name,
    c.country,
    COUNT(o.order_id) AS total_orders,
    SUM(o.sales) AS total_sales
FROM customers AS c
JOIN orders AS o
    ON o.customer_id = c.id
GROUP BY
    c.id,
    c.first_name,
    c.country
HAVING SUM(o.sales) > 600
ORDER BY total_sales DESC;
```

```
/*
    For each country, report the number of customers, the average score (rounded to
    1 decimal place), and the
    minimum and maximum score. Only include countries with at least 50 customers
*/
```

```
SELECT
    country,
    COUNT(id) AS total_customers,
    ROUND(AVG(score), 1) AS average_score,
    MIN(score) AS min_score,
    MAX(score) AS max_score
FROM customers
GROUP BY country
HAVING COUNT(*) >= 50
ORDER BY max_score DESC;
```

```
/*
    Rank customers by score within each country, with rank 1 being the highest
    score. Return the customer id,
    name, country, score, and their rank
*/
```

```
SELECT
    id,
    first_name,
    country,
    score,
    RANK() OVER(
```

```

        PARTITION BY country
        ORDER BY score DESC
    ) AS score_rank
FROM customers
ORDER BY
    country,
    score_rank;

```

-- Find all customers whose score is higher than the overall average score across all customers.

```

SELECT
    id,
    first_name,
    country,
    score
from customers
WHERE score > (
    SELECT AVG(CAST(score AS FLOAT))
    FROM customers
)
ORDER BY score DESC;

```

```

/*
    Using a CTE, calculate the total sales and number of orders
    for each calendar month. Return the year, month, total sales,
    and order count, sorted chronologically.
*/

```

```

WITH MonthlySales AS (
    SELECT
        YEAR(order_date) AS order_year,
        MONTH(order_date) AS order_month,
        sales
    FROM orders
)
SELECT
    order_year,
    order_month,
    COUNT(*) AS total_orders,
    SUM(sales) AS total_sales
FROM MonthlySales
GROUP BY
    order_year,
    order_month
ORDER BY
    order_year,
    order_month;

```

```

/*
    List every employee along with their department's average salary, and flag
    whether that employee earns above
    their department average.
*/

```

```

SELECT
    id,
    first_name,
    last_name,
    department,
    salary,
    dept_avg_salary,
    CASE
        WHEN salary > dept_avg_salary THEN 'Above Average'
        ELSE 'At or Below Average'
    END AS salary_flag
FROM (
    SELECT
        id,
        first_name,
        last_name,
        department,
        salary,
        AVG(salary) OVER (
            PARTITION BY department
        ) AS dept_avg_salary
    FROM employees
) AS employee_data
ORDER BY salary DESC;

```

```

/*
    Generate a report showing each employee's full name, their initials (e.g. 'J.S.'),
    and the length of their full name.
*/
SELECT
    id,
    CONCAT_WS(' ', first_name, last_name) AS full_name,
    LEFT(first_name, 1) + '.' + LEFT(last_name, 1) + '.' AS initials,
    LEN(CONCAT_WS(' ', first_name, last_name)) AS name_length
FROM employees

```

```

/*
    For every order, show which quarter of the year it fell in, and how many days ago it
    was placed relative to
    2022-01-01
*/
SELECT
    order_id,
    customer_id,
    order_date,
    DATEPART(QUARTER, order_date) AS order_quarter,
    DATEDIFF(DAY, order_date, '2022-01-01') AS days_before_2022
FROM orders
ORDER BY order_date;

```

```

/*
    Classify each customer into a score band: 'Low' (0-333), 'Medium' (334-666), or
    'High' (667-1000). Then count
    how many customers fall into each band.

```

```

*/
SELECT
    CASE
        WHEN score <= 333 THEN 'Low'
        WHEN score BETWEEN 334 AND 666 THEN 'Medium'
        ELSE 'High'
    END AS score_band,
    COUNT(*) AS customer_count
FROM customers
GROUP BY
    CASE
        WHEN score <= 333 THEN 'Low'
        WHEN score BETWEEN 334 AND 666 THEN 'Medium'
        ELSE 'High'
    END
ORDER BY MIN(score);

```

```

/*
    Retrieve the full name, country, score, order date, and sales amount for every order
    placed by customers
    whose score is above 700, where the order's sales amount also exceeds 500
*/

```

```

SELECT
    c.first_name,
    c.country,
    c.score,
    o.order_date,
    o.sales
FROM customers AS c
INNER JOIN orders AS o
    ON o.customer_id = c.id
WHERE
    c.score > 700
    AND o.sales > 500
ORDER BY
    o.sales DESC;

```

```

/*
    Within each department, order employees by salary ascending, then show how much more
    each employee
    earns than the colleague immediately below them in that ordering
*/

```

```

SELECT
    department,
    first_name,
    last_name,
    salary,
    salary - LAG(salary) OVER (
        PARTITION BY department
        ORDER BY salary
    ) AS gap_from_previous
FROM employees
ORDER BY
    department,
    salary;

```

```
/* For every customer who has placed at least one order, return their most recent order date.
*/
```

```
SELECT
    c.id,
    c.first_name,
    (
        SELECT MAX(o.order_date)
        FROM orders AS o
        WHERE o.customer_id = c.id
    ) AS most_recent_order
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o2
    WHERE o2.customer_id = c.id
);
```

```
/* Return the customers who have placed at least one order with sales greater than 900, without duplicating a customer row for each such order
*/
```

```
SELECT
    c.id,
    c.first_name,
    c.country
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.id
    AND o.sales > 900
)
ORDER BY c.id;
```

```
/* Find the top 3 highest-paid employees within each department.
*/
```

```
WITH RankedEmployees AS (
    SELECT
        id,
        first_name,
        last_name,
        department,
        salary,
        ROW_NUMBER() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees
)
SELECT
    department,
```

```

        first_name,
        last_name,
        salary,
        salary_rank
FROM RankedEmployees
WHERE salary_rank <= 3
ORDER BY
    department,
    salary_rank;

```

```

/*
    For every order, show what percentage of the grand total sales that single order
    represents, rounded to 2
    decimal places.
*/

```

```

SELECT
    order_id,
    customer_id,
    sales,
    ROUND(
        100.0 * sales / SUM(sales) OVER (),
        2
    ) AS pct_of_total_sales
FROM orders
ORDER BY pct_of_total_sales DESC;

```

```

/*
    For each department, report the number of employees, the total salary budget, and
    the average salary
    (rounded to 0 decimals). Only include departments with an average salary above 5000.
*/

```

```

SELECT
    department,
    COUNT(*) AS headcount,
    SUM(salary) AS total_salary_budget,
    ROUND(AVG(CAST(salary AS FLOAT)), 0) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(CAST(salary AS FLOAT)) > 5000
ORDER BY total_salary_budget DESC;

```

```

/*
    Produce a single summary row showing the count of customers in Egypt, USA, and
    Germany as three separate
    columns (a manual pivot).
*/

```

```

SELECT
    SUM(CASE
        WHEN country = 'Egypt' THEN 1
        ELSE 0
    END) AS egypt_customers,

    SUM(CASE
        WHEN country = 'USA' THEN 1
        ELSE 0
    END) AS usa_customers,

    SUM(CASE
        WHEN country = 'Germany' THEN 1
        ELSE 0
    END) AS germany_customers;

```

```
END) AS usa_customers,  
SUM(CASE  
  WHEN country = 'Germany' THEN 1  
  ELSE 0  
END) AS germany_customers  
FROM customers;
```